

# GitStory

## Software Requirements Specification

---

**Prepared by:**

Ahmad Muaz Asad — 23L-0844

Abdul Munam Aziz — 23L-3019

Bilal Ahmad Khan — 23L-0872

Junaid Muhammad — 23L-1014

Sabih Ud Din Khan — 23L-0654

Department of Computer Science

Date: March 2026

# Table of Contents

|                                               |           |
|-----------------------------------------------|-----------|
| <b>1. Introduction</b>                        | <b>3</b>  |
| 1.1 Purpose                                   | 3         |
| 1.2 Document Conventions                      | 3         |
| 1.3 Intended Audience and Reading Suggestions | 3         |
| 1.4 Product Scope                             | 4         |
| 1.5 References                                | 4         |
| <b>2. Overall Description</b>                 | <b>5</b>  |
| 2.1 Product Perspective                       | 5         |
| 2.2 Product Functions                         | 6         |
| 2.2.1 Context Diagram (DFD Level 0)           | 6         |
| 2.2.2 Use Case Diagrams                       | 7         |
| 2.2.3 Summary of Core Functions               | 9         |
| 2.3 User Classes and Characteristics          | 9         |
| 2.4 Operating Environment                     | 10        |
| 2.5 Design and Implementation Constraints     | 10        |
| 2.6 User Documentation                        | 10        |
| 2.7 Assumptions and Dependencies              | 10        |
| <b>3. External Interface Requirements</b>     | <b>11</b> |
| 3.1 User Interfaces                           | 11        |
| 3.1.1 Landing Screen                          | 11        |
| 3.1.2 Login / Authentication Screen           | 12        |
| 3.1.3 Sign Up Screen via Email                | 13        |
| 3.1.4 Main Dashboard                          | 14        |
| 3.1.5 Project Stats Screen                    | 15        |
| 3.1.6 Timeline Narration Screen               | 16        |
| 3.1.7 Collaborators Screen                    | 17        |
| 3.1.8 AI Code Review Screen                   | 18        |
| 3.1.9 RAG Chat Interface                      | 19        |
| 3.2.0 File Hotzones Screen                    | 20        |
| 3.2 Hardware Interfaces                       | 20        |
| 3.3 Software Interfaces                       | 21        |
| 3.4 Communications Interfaces                 | 21        |
|                                               | 21        |

|                                                |           |
|------------------------------------------------|-----------|
| <b>4. System Features</b>                      | <b>22</b> |
| 4.1 User Authentication and Profile Management | 22        |
| 4.1.1 Description                              | 22        |
| 4.1.2 Stimulus/Response Sequences              | 22        |
| 4.1.3 Functional Requirements                  | 22        |
| 4.2 Email Registration and Authentication      | 23        |
| 4.2.1 Description                              | 23        |
| 4.2.2 Stimulus/Response Sequences              | 23        |
| 4.2.3 Functional Requirements                  | 23        |
| 4.3 Project Statistics Dashboard               | 24        |
| 4.3.1 Description                              | 24        |
| 4.3.2 Stimulus/Response Sequences              | 24        |
| 4.2.3 Functional Requirements                  | 24        |
| 4.4 Timeline Narration                         | 25        |
| 4.4.1 Description                              | 25        |
| 4.4.2 Stimulus/Response Sequences              | 25        |
| 4.4.3 Functional Requirements                  | 26        |
| 4.5 Collaborator Categorization and Awards     | 26        |
| 4.5.1 Description                              | 26        |
| 4.5.2 Stimulus/Response Sequences              | 26        |
| 4.5.3 Functional Requirements                  | 26        |
| 4.6 AI Code Review                             | 27        |
| 4.6.1 Description                              | 27        |
| 4.6.2 Stimulus/Response Sequences              | 27        |
| 4.6.3 Functional Requirements                  | 27        |
| 4.7 RAG-Based Repository Querying              | 28        |
| 4.7.1 Description                              | 28        |
| 4.7.2 Stimulus/Response Sequences              | 28        |
| 4.7.3 Functional Requirements                  | 28        |
| 4.8 Auto-Documentation                         | 29        |
| 4.8.1 Description                              | 29        |
| 4.8.2 Stimulus/Response Sequences              | 29        |
| 4.8.3 Functional Requirements                  | 30        |
| 4.9 File Hotzones (Heatmap)                    | 30        |
| 4.9.1 Description                              | 30        |
| 4.9.2 Stimulus/Response Sequences              | 30        |
| 4.9.3 Functional Requirements                  |           |

|                                                |           |
|------------------------------------------------|-----------|
| <b>5. Other Non-Functional Requirements</b>    | <b>31</b> |
| 5.1 Performance Requirements                   | 31        |
| 5.2 Safety Requirements                        | 31        |
| 5.3 Security Requirements                      | 32        |
| 5.4 Software Quality Attributes                | 32        |
| 5.5 Business Rules                             | 32        |
| <b>6. Other Requirements</b>                   | <b>34</b> |
| 6.1 Database Requirements                      | 34        |
| 6.2 Internationalization                       | 34        |
| 6.3 Legal and Compliance                       | 34        |
| <b>Appendix A: Glossary</b>                    | <b>35</b> |
| <b>Appendix B: Analysis Models</b>             | <b>36</b> |
| B.1 Design Class Diagram                       | 36        |
| B.2 DFD Level 1 — Detailed Data Flow Diagram   | 37        |
| <b>Appendix C: To Be Determined (TBD) List</b> | <b>38</b> |

# 1. Introduction

---

## 1.1 Purpose

This Software Requirements Specification (SRS) document describes the complete functional and non-functional requirements for GitStory — an AI-powered Git repository analysis and narration platform. It serves as the authoritative reference for the development team, academic evaluators, and stakeholders throughout the development lifecycle.

GitStory is targeted at Version 1.0 and covers the full scope of its seven core analytical features: Project Stats, Timeline Narration, Collaborator Categorization and Awards, AI Code Review, RAG Querying, Auto-Documentation, and File Hotzones.

## 1.2 Document Conventions

This document follows the IEEE Std 830-1998 SRS template. The following conventions apply throughout:

- Requirements are labeled REQ-XX-YY, where XX represents the feature area and YY is a sequential number.
- Priority levels are defined as High, Medium, or Low.
- Bold text is used for key terms and emphasis.
- SHALL indicates a mandatory requirement; SHOULD indicates a recommended requirement; MAY indicates an optional requirement.
- TBD marks items yet to be determined.

## 1.3 Intended Audience and Reading Suggestions

This document is intended for the following audiences:

| Audience                   | Recommended Sections                                         |
|----------------------------|--------------------------------------------------------------|
| <b>Developers</b>          | All sections, with emphasis on Sections 3, 4, and Appendix B |
| <b>Project Managers</b>    | Sections 1, 2, and 5                                         |
| <b>Testers / QA</b>        | Sections 4 and 5                                             |
| <b>Academic Evaluators</b> | Sections 1, 2, and Appendix B                                |
| <b>End Users</b>           | Sections 2 and 3                                             |

## 1.4 Product Scope

GitStory is a web-based platform that performs a comprehensive multi-dimensional analysis of Git repositories. It addresses the "knowledge rot" problem — the loss of engineering intent as codebases grow — by converting raw version-control data into actionable narratives, visual insights, and intelligent answers.

The system covers seven analysis dimensions:

- Analytics: Quantitative metrics and visualization of project activity.
- Narration: AI-generated story-like summaries of commit history.
- Team Dynamics: Behavioral profiling and ranking of collaborators.
- Quality: Automated code review with health scoring.
- Intelligence: Natural language querying over the codebase (RAG).
- Documentation: Automated README and function-level documentation generation.
- Risk Visualization: Heatmap highlighting files prone to churn and instability.

## 1.5 References

- GitStory Project Proposal Document, Team GitStory, 2026.
- IEEE Std 830-1998 — Recommended Practice for Software Requirements Specifications.
- PyDriller Documentation — <https://pydriller.readthedocs.io>
- FastAPI Documentation — <https://fastapi.tiangolo.com>
- OpenRouter API Documentation — <https://openrouter.ai/docs>
- Pinecone Vector Database Documentation — <https://docs.pinecone.io>
- Next.js Documentation — <https://nextjs.org/docs>
- Supabase Documentation — <https://supabase.com/docs>

## 2. Overall Description

---

### 2.1 Product Perspective

GitStory is a standalone web application that integrates with GitHub through its OAuth API. It does not replace Git or GitHub; rather, it sits on top of Git history as an intelligent post-processing and analysis layer.

The system interacts with three major external systems:

- GitHub OAuth API: For user authentication and repository access.
- OpenRouter LLM API: For AI-powered narration, code review, documentation generation, and RAG querying.
- Pinecone / ChromaDB Vector Database: For storing and retrieving code embeddings used in the RAG interface.

GitStory is composed of three primary subsystems: the Frontend Dashboard (Next.js), the Backend Analysis Engine (FastAPI + Python), and the AI/Intelligence Layer (LLM + Vector DB). These are connected via RESTful APIs over HTTPS.

## 2.2 Product Functions

The following subsections describe the major product functions. Diagrams are included below for reference.

### 2.2.1 Context Diagram (DFD Level 0)

The Context Diagram below shows the GitStory system and all external entities it interacts with. The system receives a GitHub repository link and natural language queries from the user, fetches source code and commit metadata from GitHub, sends formatted prompts to the LLM API, and returns dashboards, narratives, and RAG responses to the user.

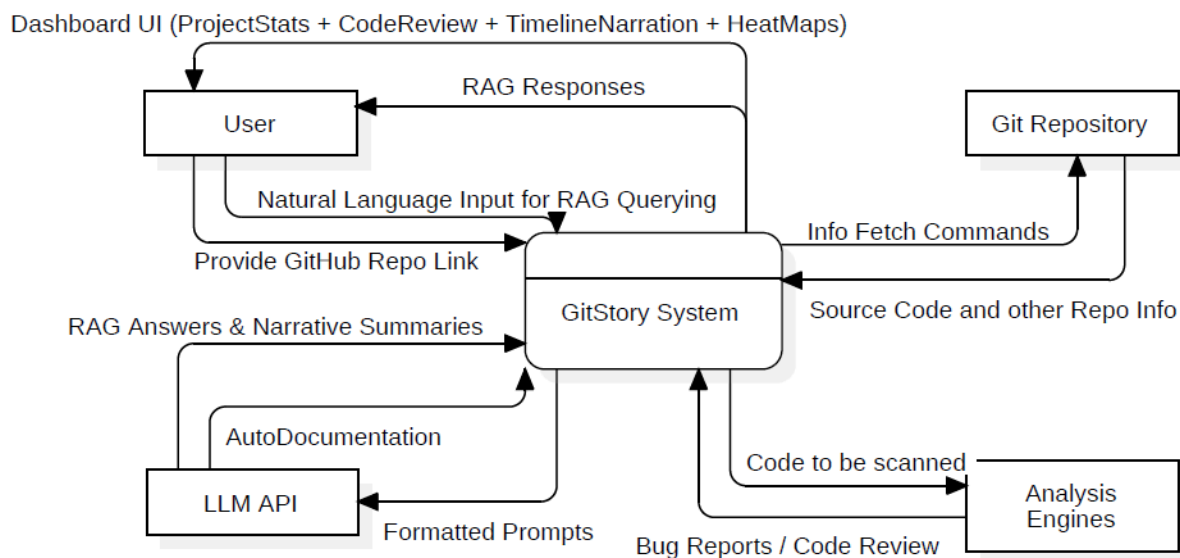


Figure 2.1: GitStory System Context Diagram (DFD Level 0) — Shows the system boundary and external entity interactions including User, Git Repository, and LLM API.

Key external data flows in the Context Diagram:

- User → System: GitHub repository URL, natural language queries for RAG, authentication credentials.
- System → User: Dashboard UI with Project Stats, Code Review results, Timeline Narration, Heatmaps, RAG responses, and Auto-Documentation.
- System ↔ Git Repository: Source code retrieval, commit history, contributor metadata.
- System ↔ LLM API: Formatted prompts sent to the LLM; narrative summaries, code review reports, and documentation received in return.
- System ↔ Vector Database: Codebase embeddings stored and semantic context retrieved for RAG.



## 2.2.2 Use Case Diagrams

GitStory's use cases are organized into three subsystems, each described below.

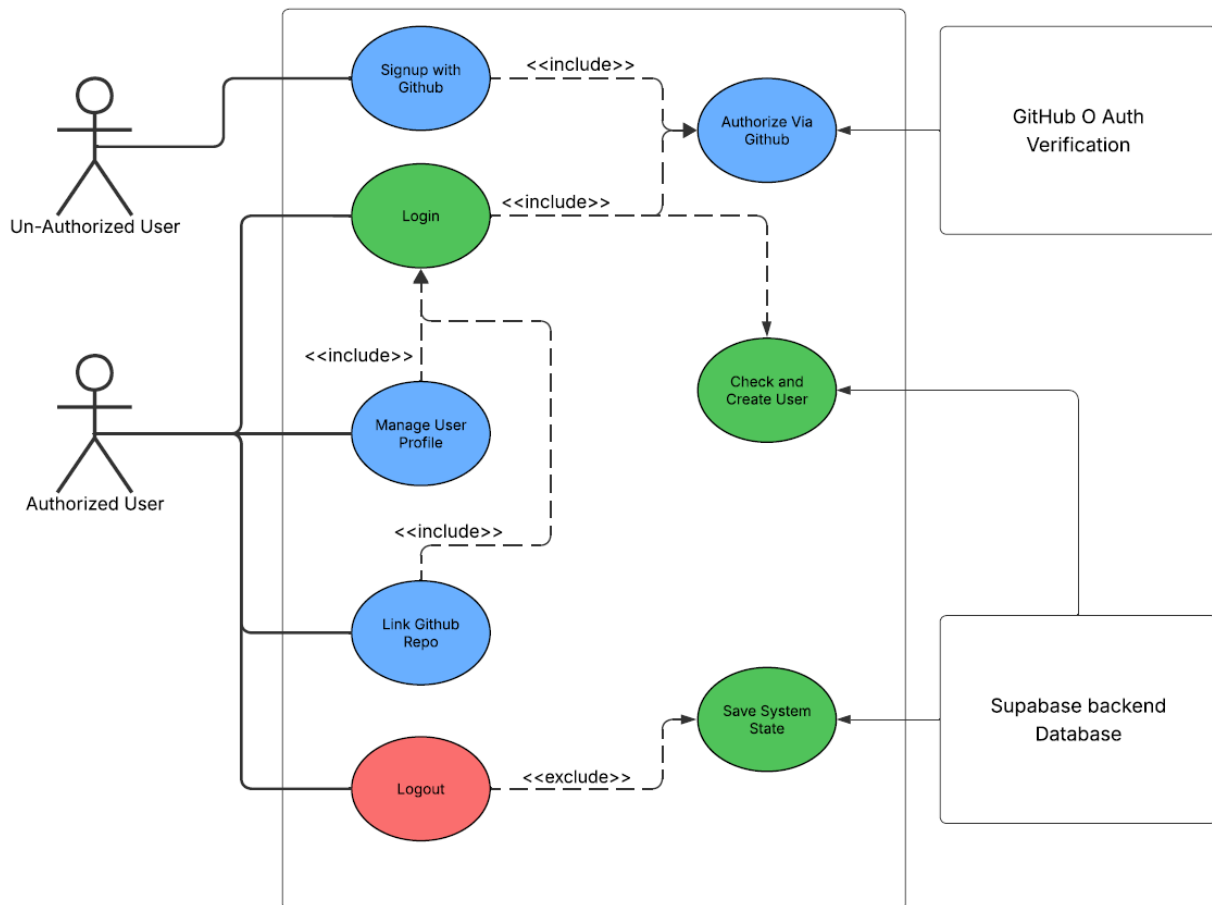


Figure 2.2a: Authentication Use Case Diagram — Covers GitHub OAuth signup/login, profile management, repository linking, and Supabase session management.

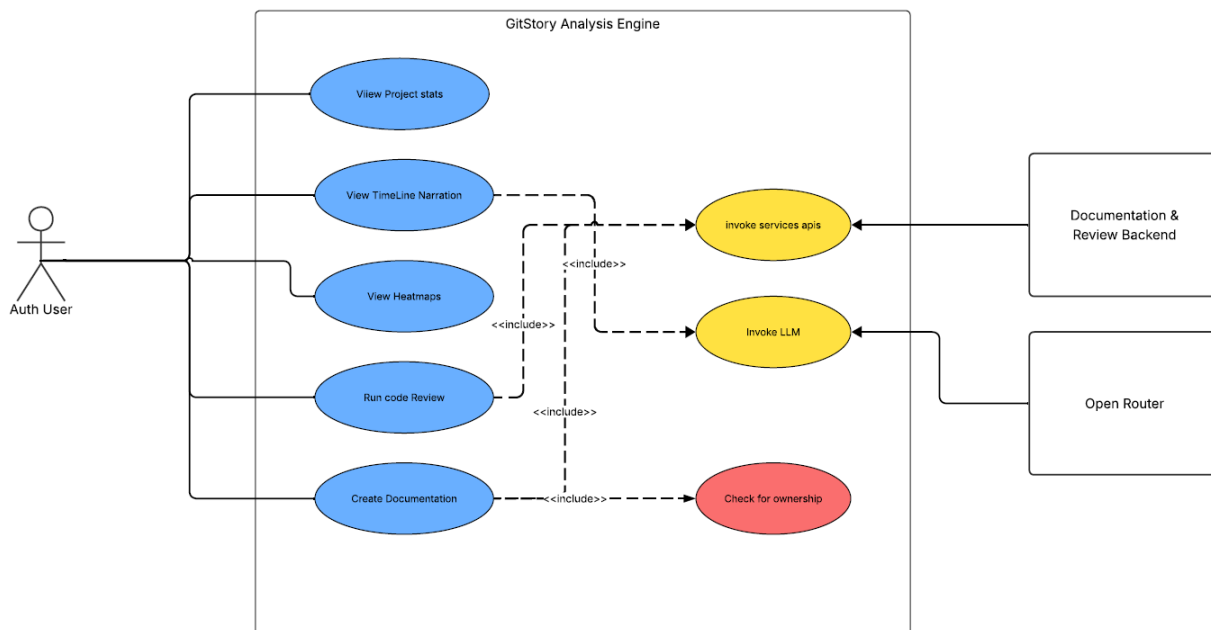


Figure 2.2b: Analysis Engine Use Case Diagram — Covers Project Stats, Timeline Narration, Heatmap viewing, Code Review execution, and Auto-Documentation creation.



Figure 2.2c: Intelligent Features Use Case Diagram — Covers RAG-based chat querying via Pinecone, Collaborator Awards viewing, and individual award details.

### 2.2.3 Summary of Core Functions

The seven core product functions are summarized in the table below:

| # | Feature                    | Description                                                                                                        | Priority |
|---|----------------------------|--------------------------------------------------------------------------------------------------------------------|----------|
| 1 | <b>Project Stats</b>       | Dashboard showing commit frequency, language distribution, pull-request velocity, and contribution charts.         | High     |
| 2 | <b>Timeline Narration</b>  | AI-driven conversion of commit history into a readable, story-like chronological narrative.                        | High     |
| 3 | <b>Collaborator Awards</b> | Behavioral categorization of contributors (e.g., 'Feature Lead', 'Night Owl') and ranking based on preset metrics. | Medium   |
| 4 | <b>AI Code Review</b>      | Automated scan of commits for bugs and best-practice violations, with a codebase health score.                     | High     |
| 5 | <b>RAG Querying</b>        | Natural language chat interface to query the repository using Retrieval-Augmented Generation.                      | High     |
| 6 | <b>Auto-Documentation</b>  | Automated generation of READMEs and function-level summaries by parsing the code structure.                        | Medium   |
| 7 | <b>File Hotzones</b>       | Treemap/heatmap visualization highlighting frequently modified files (churn) and risk zones.                       | Medium   |

## 2.3 User Classes and Characteristics

| User Class                     | Description                                                                                                                                           | Technical Level                      |
|--------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------|
| <b>Developer / Contributor</b> | A software engineer who has committed to a GitHub repository and wants to review their own work, understand project history, or improve code quality. | High — comfortable with code and Git |
| <b>Student</b>                 | A university student using GitStory to present their group project's development journey and individual contribution during academic reviews.         | Medium — familiar with Git basics    |
| <b>Instructor / Evaluator</b>  | An academic who uses GitStory to objectively assess individual contributions in group projects based on behavioral profiling and commit analytics.    | Low to Medium — non-developer        |
| <b>Team Lead / Manager</b>     | A technical lead who wants a bird's-eye view of team productivity, code health, and risk areas in the codebase.                                       | High — strategic and technical       |

## 2.4 Operating Environment

- Client: Any modern web browser (Chrome 100+, Firefox 100+, Safari 15+, Edge 100+) on desktop or laptop.
- Server: Python 3.10+ with FastAPI running on a Linux-based server (Ubuntu 22.04 recommended).
- Database: PostgreSQL via Supabase (cloud-hosted).
- Vector Store: Pinecone or ChromaDB (cloud or self-hosted).
- AI Services: OpenRouter API — requires a stable internet connection.
- Internet: The backend must have outbound access to GitHub API, OpenRouter API, and Pinecone/ChromaDB.

## 2.5 Design and Implementation Constraints

- GitHub-only: The system supports GitHub repositories exclusively. SVN, GitLab, and Bitbucket are out of scope for v1.0.
- API Rate Limits: GitHub REST API rate limits (5,000 requests/hour for authenticated users) may constrain large repository analysis.
- LLM Cost: OpenRouter API calls incur per-token costs, which limits batch processing size and frequency.
- Language Detection: Automatic language detection is limited to languages supported by PyDriller and GitHub Linguist.
- Private Repositories: The OAuth token scope must include 'repo' permission for private repository access.
- Real-time IDE integration is explicitly out of scope; GitStory is a post-processing tool.

## 2.6 User Documentation

- In-app onboarding tour using step-by-step tooltips (built into the Next.js frontend).
- Contextual help icons on each dashboard section linking to a short explanation.
- README.md for developers covering API setup, environment variables, and local development.
- TBD: User manual PDF to be delivered with v1.0 release.

## 2.7 Assumptions and Dependencies

- Users have an active GitHub account and at least one repository with commit history.
- OpenRouter API keys are available and properly configured in the server environment.
- Supabase project is provisioned with the correct schema prior to deployment.
- The development machine has Node.js 18+, Python 3.10+, and npm installed.
- Internet connectivity is required for all AI features (narration, RAG, code review, documentation).
- PyDriller library continues to be maintained and compatible with GitHub's repository format.

## 3. External Interface Requirements

---

### 3.1 User Interfaces

GitStory's frontend is built with Next.js and uses Recharts for standard charts and D3.js for complex visualizations (heatmaps, tree maps). The following UI mockup placeholders describe each primary screen. Actual Figma wireframes will be inserted into this section upon completion of the UI design phase.

#### 3.1.1 Landing Screen

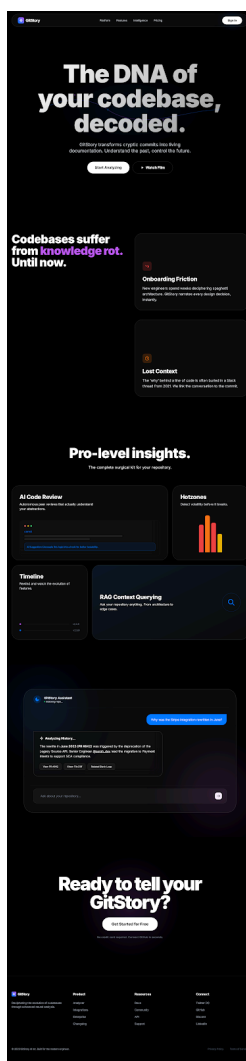
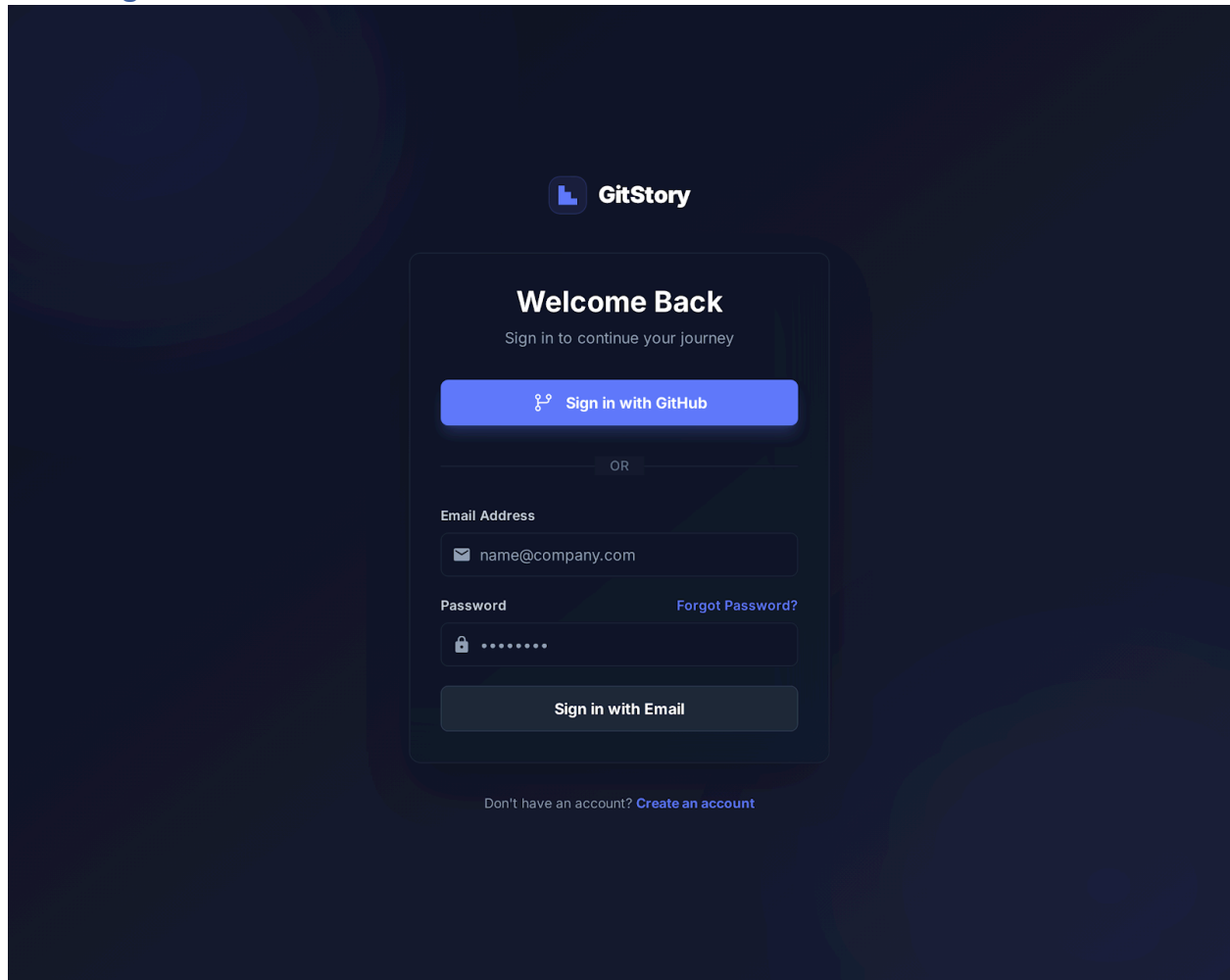


Figure 3.1: The landing page contains a top navigation bar with the GitStory logo and a sign-in button; a cinematic hero section with a bold headline and 3D visual elements; and an asymmetrical grid of feature cards highlighting the core platform capabilities.

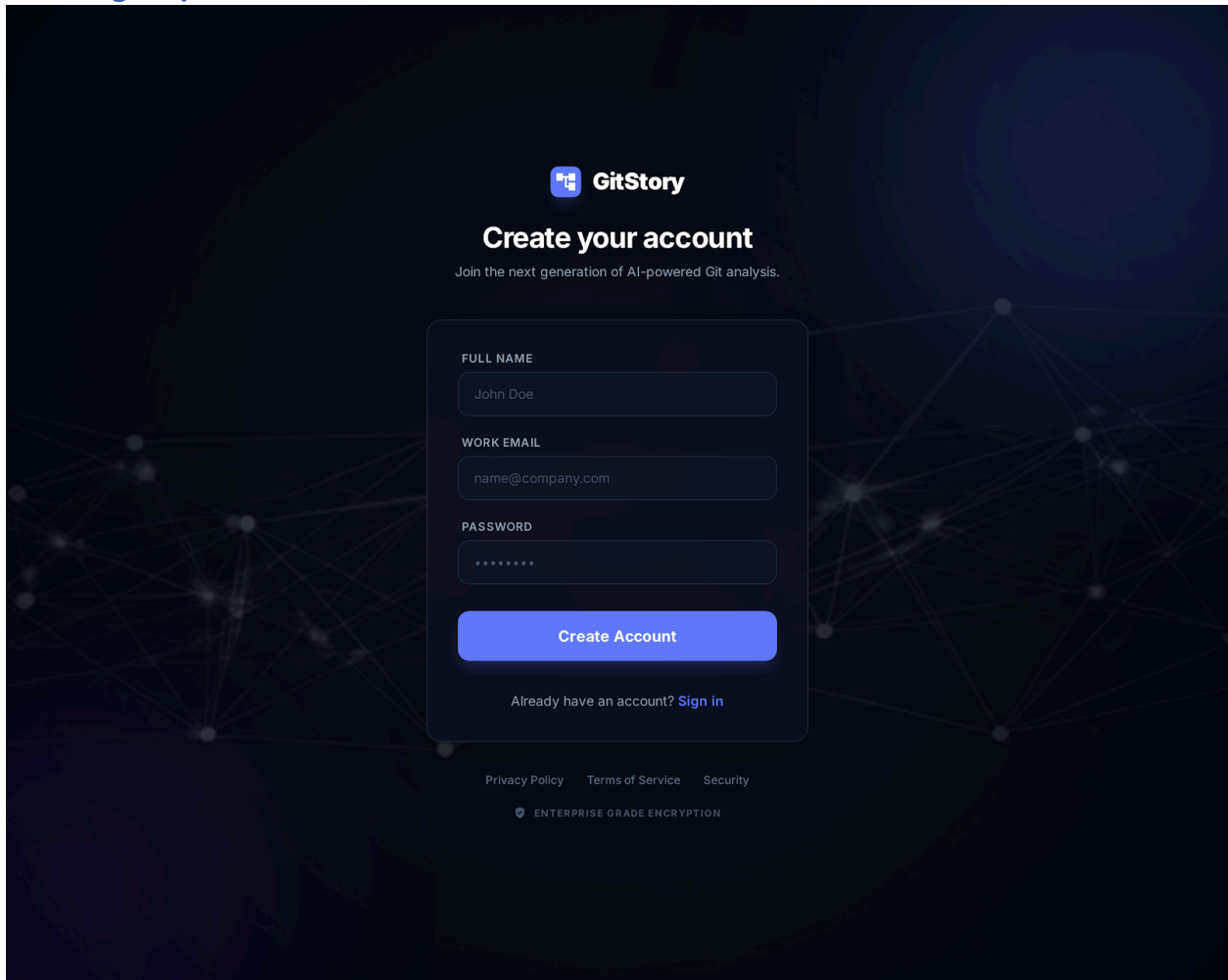
### 3.1.2 Login / Authentication Screen



*Figure 3.1: Login Screen — GitHub OAuth button, GitStory branding, and brief tagline.*

The login screen presents a centered card with the GitStory logo, a 'Sign in with GitHub' OAuth button, and a short tagline. No username/password entry is required. Upon clicking, the user is redirected to GitHub for OAuth authorization and then returned to the dashboard.

### 3.1.3 Sign Up Screen via Email



The image shows a dark-themed sign-up screen for GitStory. At the top center is the GitStory logo, which consists of a blue square with a white 'G' and the text 'GitStory' in white. Below the logo is the heading 'Create your account' in white, followed by the subtext 'Join the next generation of AI-powered Git analysis.' in a smaller white font. The main content is a centered white card with rounded corners. Inside the card, there are three input fields: 'FULL NAME' with the placeholder 'John Doe', 'WORK EMAIL' with the placeholder 'name@company.com', and 'PASSWORD' with a masked password '\*\*\*\*\*'. Below these fields is a large blue button with the text 'Create Account' in white. At the bottom of the card, there is a link 'Already have an account? Sign in' in blue. Below the card, there are three links: 'Privacy Policy', 'Terms of Service', and 'Security'. At the very bottom, there is a small shield icon followed by the text 'ENTERPRISE GRADE ENCRYPTION'.

Figure 3.1: The sign-up screen contains: a centered card with the GitStory logo and an account creation heading; a 'Sign up with GitHub' OAuth button; a visual divider; and a registration form with email, password, and confirm password input fields. A text link at the bottom directs existing users back to the sign-in page.

### 3.1.4 Main Dashboard

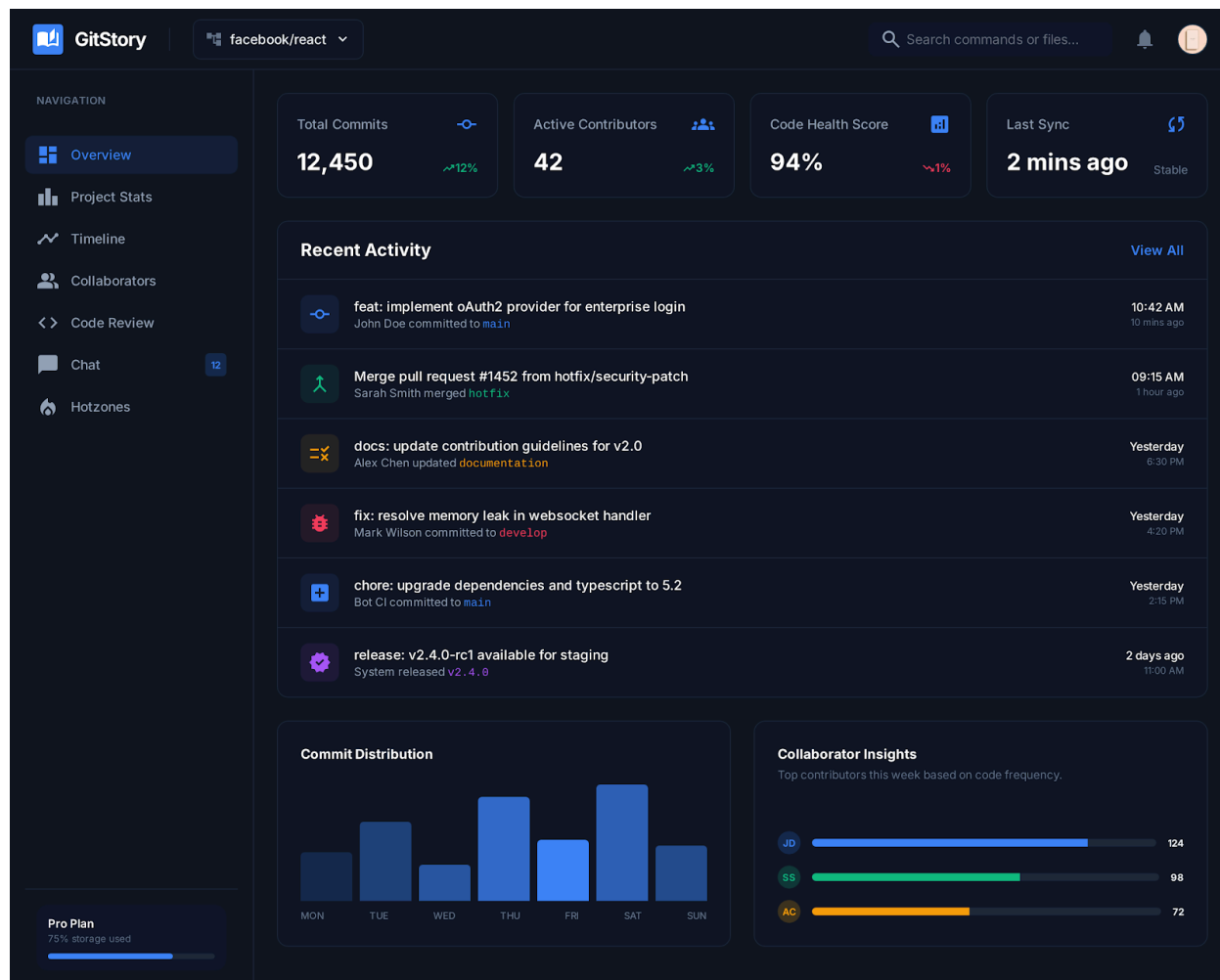


Figure 3.2: Main Dashboard — Repository selector, navigation sidebar, and summary metric cards.

The dashboard contains: a top navigation bar with the user avatar and a repository selector dropdown; a left sidebar with links to each of the seven features; and a main content area. Summary cards at the top show total commits, active contributors, health score, and last sync time.



### 3.1.5 Project Stats Screen

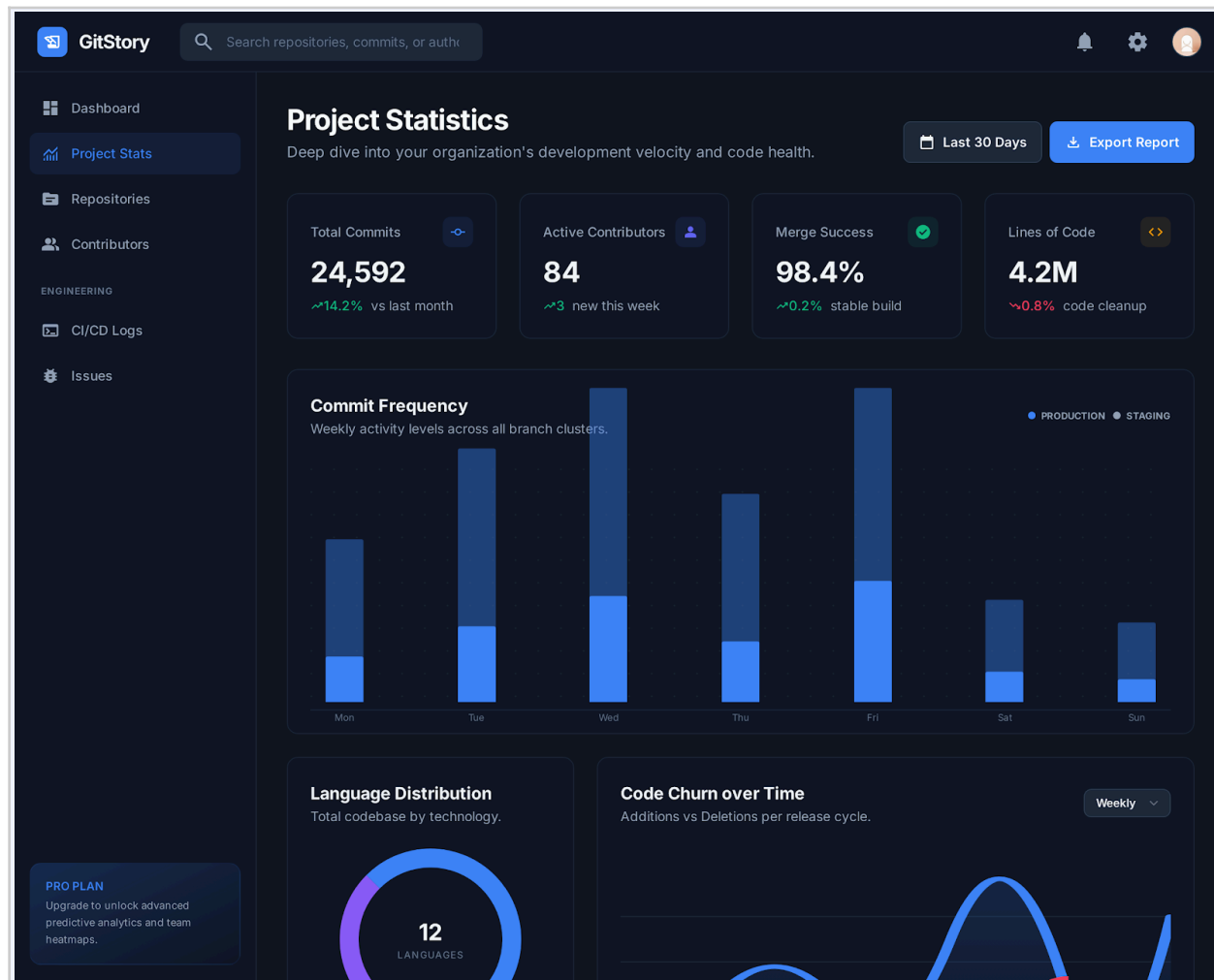


Figure 3.3: Project Stats Screen — Commit frequency bar chart, language distribution pie chart, PR velocity line chart.

### 3.1.6 Timeline Narration Screen

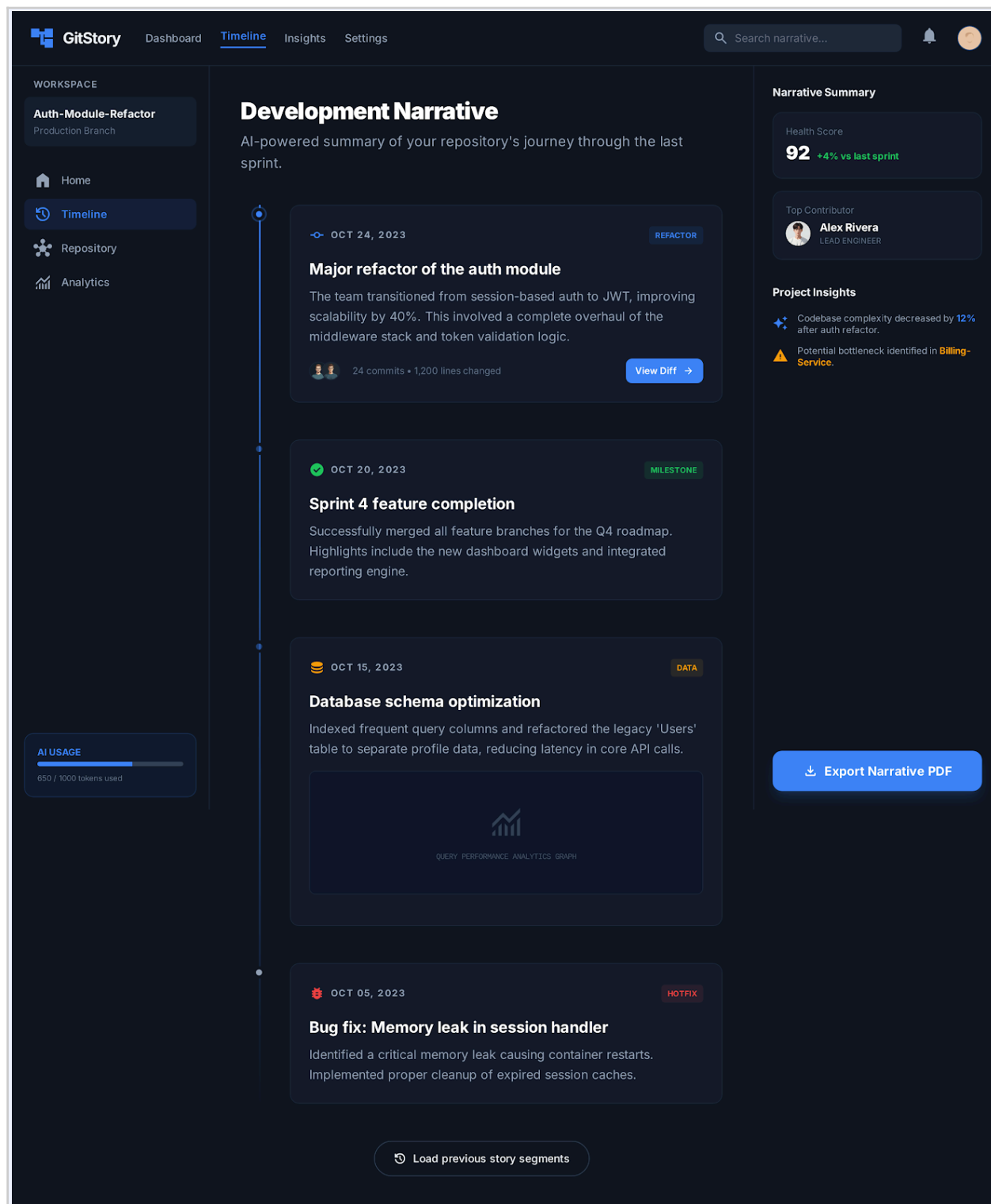


Figure 3.4: Timeline Narration Screen — D3.js scrollable timeline with AI-generated story segments and date markers.

### 3.1.7 Collaborators Screen

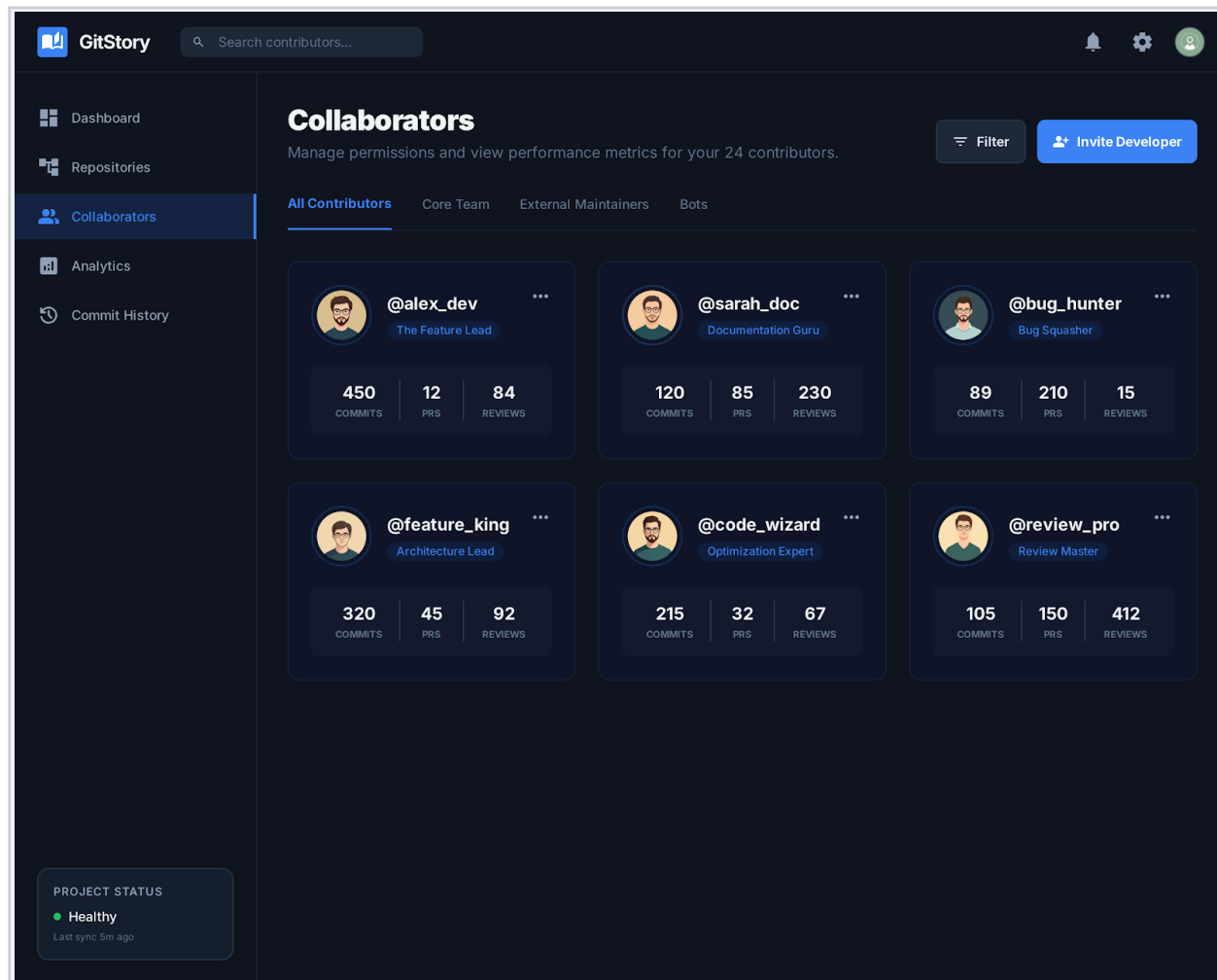


Figure 3.5: Collaborators Screen — Developer cards showing role badge, rank, commit count, and awarded titles.

### 3.1.8 AI Code Review Screen

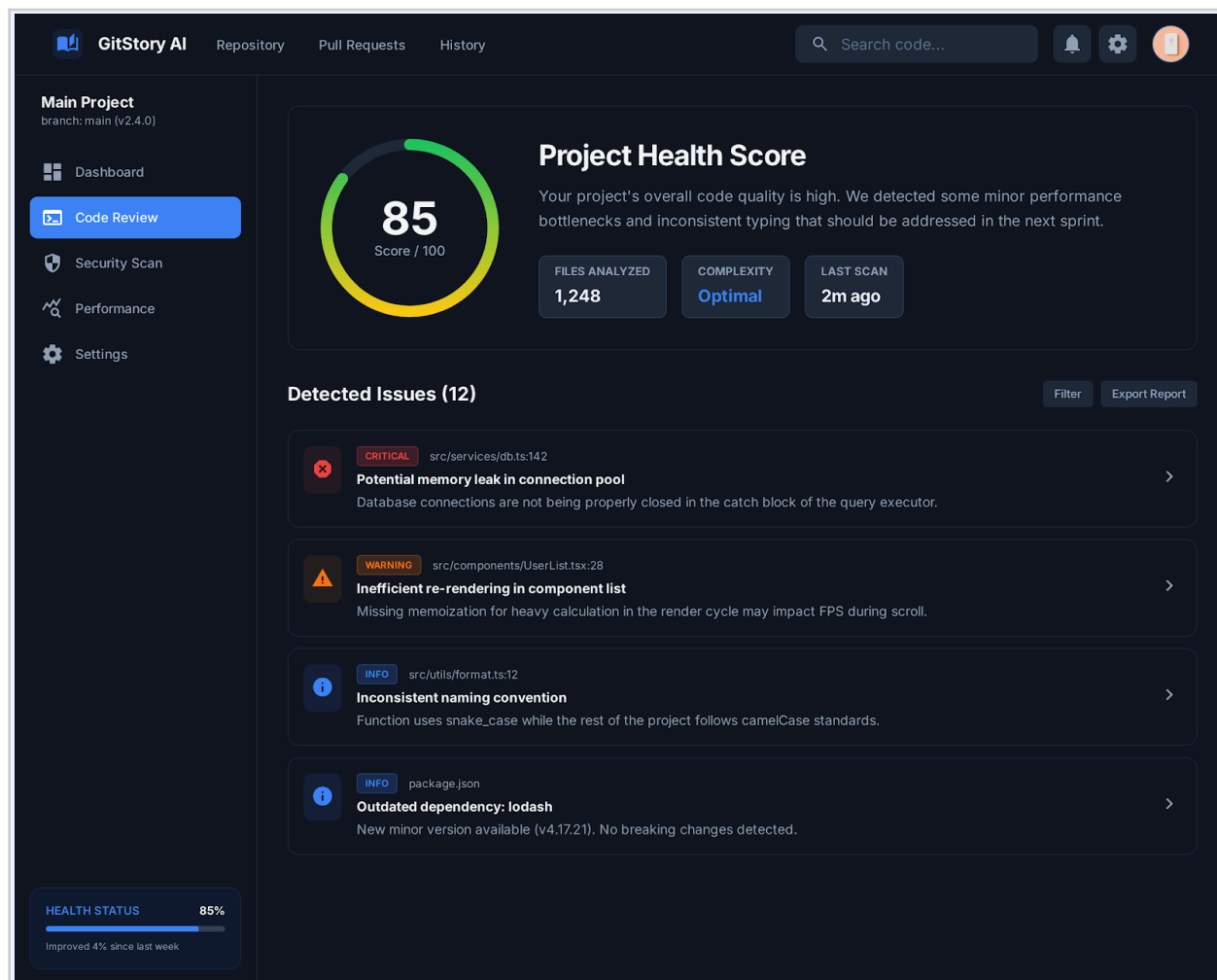


Figure 3.6: AI Code Review Screen — Health score gauge, issues list with severity tags, and suggestions panel.

### 3.1.9 RAG Chat Interface

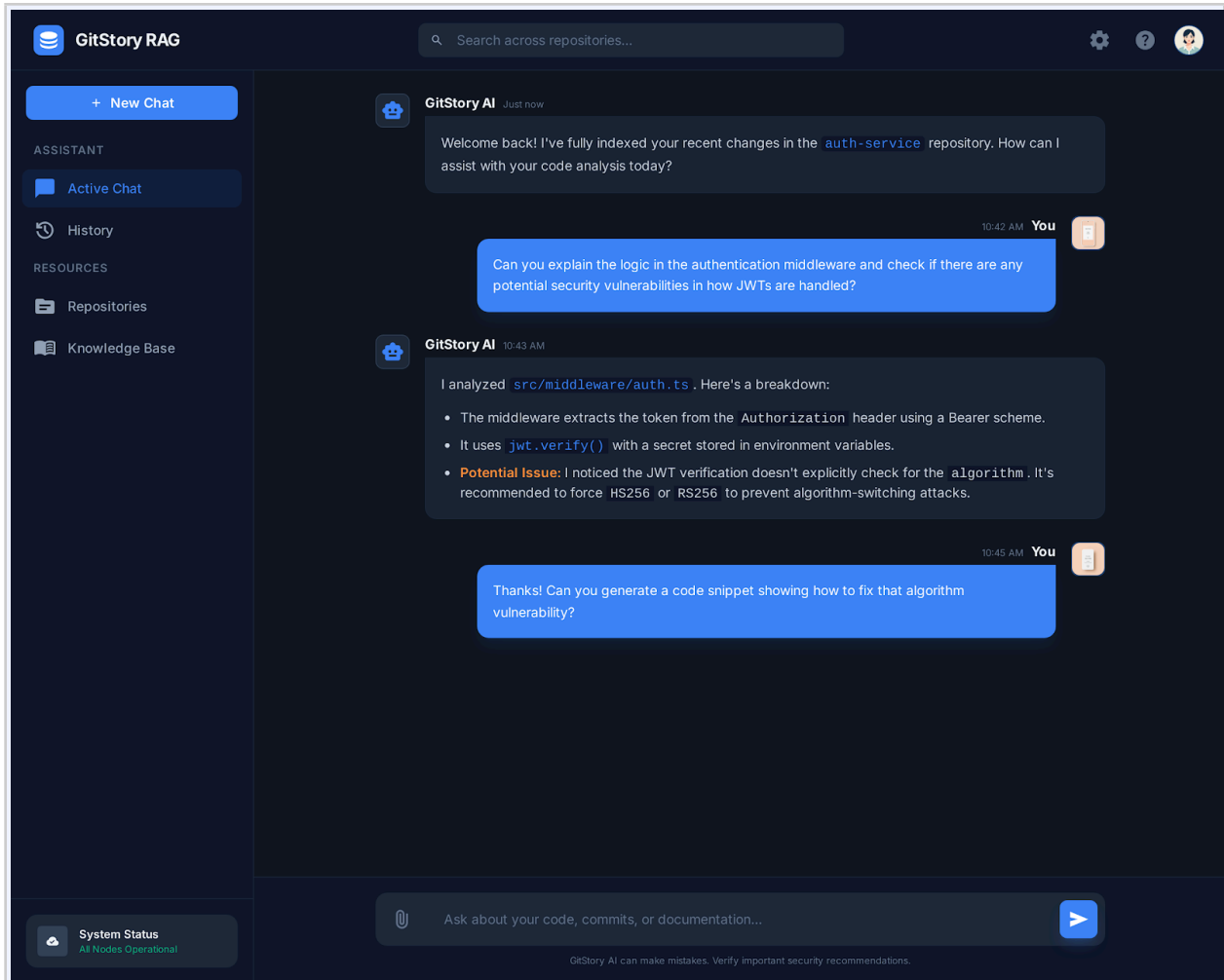


Figure 3.7: RAG Chat Interface — Conversational chat panel with message bubbles, source citation links, and input box.

### 3.2.0 File Hotzones Screen

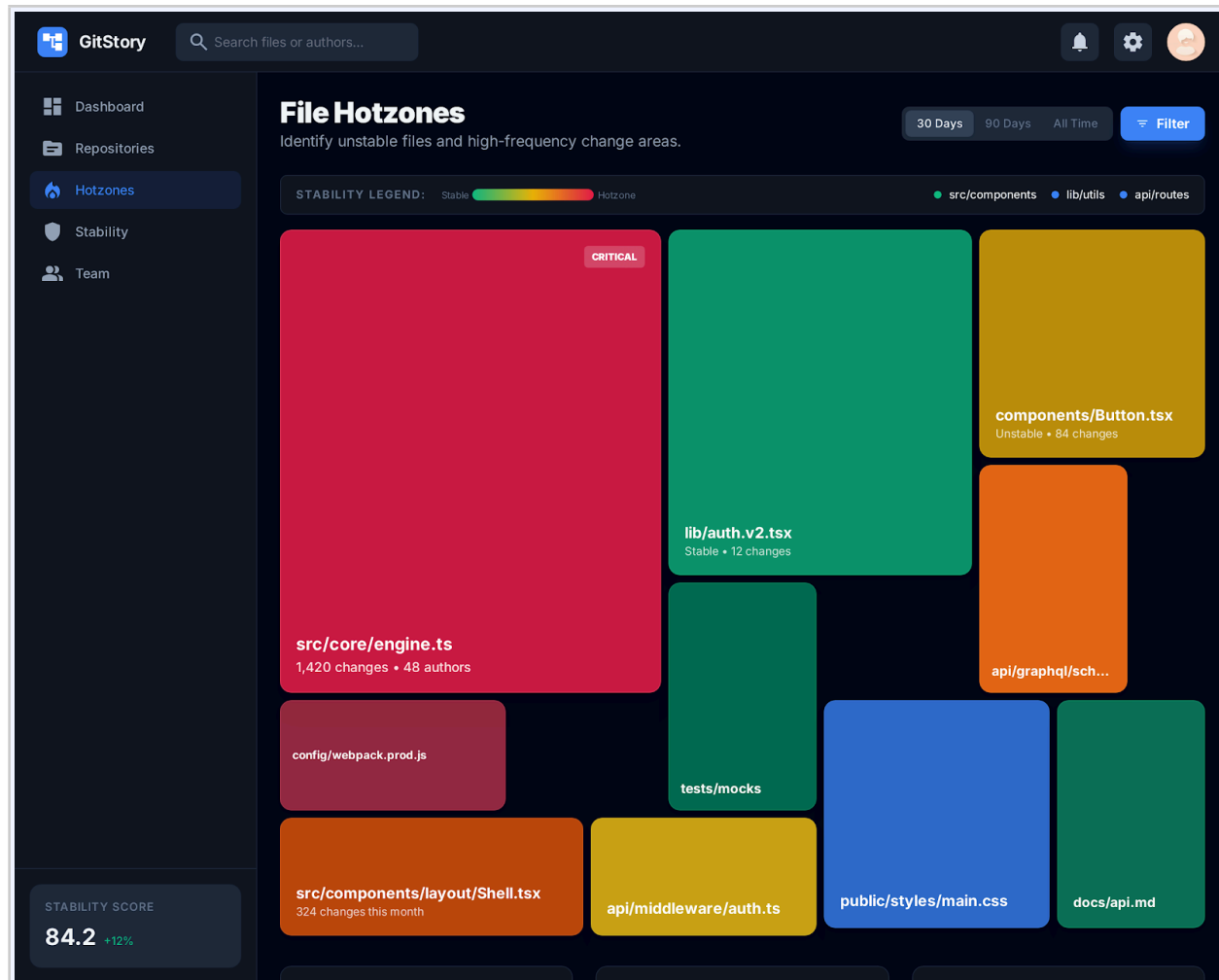


Figure 3.8: File Hotzones — D3.js Treemap visualization with red (high churn) to green (stable) color coding.

Note: All wireframes above are placeholders. Final Figma designs will be inserted in a future revision of this document after the UI design sprint.

### 3.2 Hardware Interfaces

GitStory does not interact with any physical hardware devices. It is a web-based application that runs entirely on standard server and client hardware. The application requires:

- Server: A machine with at least 4 CPU cores, 8 GB RAM, and 50 GB SSD storage.
- Client: A device capable of running a modern web browser with a minimum screen resolution of 1280x720.
- Network: HTTPS (port 443) connectivity for API communication.

### 3.3 Software Interfaces

| Software Component    | Version / Type      | Purpose                                                                                         |
|-----------------------|---------------------|-------------------------------------------------------------------------------------------------|
| GitHub REST API       | v3 / HTTPS REST     | OAuth authentication, repository cloning, commit and contributor metadata retrieval             |
| OpenRouter API        | Latest / HTTPS REST | LLM calls for narration, code review, documentation generation, and RAG synthesis               |
| Pinecone / ChromaDB   | Latest              | Storage and semantic retrieval of codebase embeddings for the RAG interface                     |
| Supabase (PostgreSQL) | Latest              | Persistent storage for user profiles, project metadata, analysis results, and collaborator data |
| PyDriller             | 2.x                 | Python library for mining Git history, extracting commit data, and computing file churn metrics |
| Next.js               | 14.x                | Frontend framework for the responsive dashboard UI                                              |
| FastAPI               | 0.100+              | Python backend framework for RESTful API endpoints                                              |
| Recharts              | Latest              | Standard chart rendering (bar, line, pie) for project statistics                                |
| D3.js                 | 7.x                 | Complex visualizations: Treemap heatmap and Timeline Narration stream                           |

### 3.4 Communications Interfaces

- All client-server communication is over HTTPS (TLS 1.2+) using JSON payloads.
- GitHub OAuth 2.0 flow is used for authentication; access tokens are stored securely in the Supabase session, never in local storage.
- WebSockets (or Server-Sent Events) may be used for streaming AI-generated narration and code review results to the frontend in real time.
- The backend communicates with OpenRouter API over HTTPS REST with Bearer token authentication.
- Pinecone/ChromaDB connections use their respective SDK clients over HTTPS.

## 4. System Features

### 4.1 User Authentication and Profile Management

**Priority:** High

**Benefit:** 9 | **Penalty:** 9 | **Cost:** 3 | **Risk:** 2

#### 4.1.1 Description

This feature provides secure, GitHub-based authentication so that only authorized users can access GitStory. Users sign up and log in via GitHub OAuth 2.0. Their profile data (avatar, username, GitHub ID, linked repositories) is persisted in Supabase. Users can manage their profile and link/unlink GitHub repositories.

#### 4.1.2 Stimulus/Response Sequences

- User clicks 'Sign in with GitHub' → System redirects to GitHub OAuth → User authorizes → System creates/retrieves user record in Supabase → User lands on dashboard.
- Authenticated user navigates to Profile → System retrieves profile data from Supabase → User views avatar, username, and linked repos.
- User links a new repository → System validates ownership → Repository record created in database.
- User logs out → System invalidates session in Supabase → User redirected to login screen.

#### 4.1.3 Functional Requirements

| Req. ID   | Requirement Description                                                                                                                 | Priority |
|-----------|-----------------------------------------------------------------------------------------------------------------------------------------|----------|
| REQ-AU-01 | The system SHALL allow users to sign up and log in exclusively via GitHub OAuth 2.0.                                                    | High     |
| REQ-AU-02 | Upon first login, the system SHALL create a new user record in Supabase storing: user UUID, GitHub ID, username, email, and avatar URL. | High     |
| REQ-AU-03 | The system SHALL maintain a secure session using Supabase Auth and store the GitHub access token server-side only.                      | High     |
| REQ-AU-04 | Users SHALL be able to view and update their display name and avatar from the profile page.                                             | Medium   |
| REQ-AU-05 | Users SHALL be able to link one or more GitHub repositories to their GitStory account.                                                  | High     |
| REQ-AU-06 | The system SHALL verify repository ownership before allowing it to be linked.                                                           | High     |
| REQ-AU-07 | The system SHALL support logout, which invalidates the session token in Supabase.                                                       | High     |



| Req. ID   | Requirement Description                                                                                                                          | Priority |
|-----------|--------------------------------------------------------------------------------------------------------------------------------------------------|----------|
| REQ-AU-08 | The system SHALL allow users to register and log in using a standard email address and password combination.                                     | High     |
| REQ-AU-09 | The system SHALL provide a "Forgot Password" mechanism allowing users to securely reset their password via an email link.                        | High     |
| REQ-LP-01 | The system SHALL provide an unauthenticated public landing page detailing the product's features and directing users to the authentication flow. | Medium   |

## 4.2 Email Registration and Authentication

**Priority:** High

**Benefit:** 9 | **Penalty:** 8 | **Cost:** 4 | **Risk:** 5

### 4.2.1 Description

This feature provides an alternative, secure authentication method, allowing users to create an account and log in using a standard email address and password. It ensures platform accessibility for users who prefer not to use GitHub OAuth for their primary account creation. User credentials are encrypted, and sessions are managed securely via Supabase Auth.

### 4.2.2 Stimulus/Response Sequences

- User navigates to the Sign-Up screen, enters a valid email and password, and clicks submit → System validates the input, creates a new user record in Supabase Auth, and securely hashes the password → User is authenticated and redirected to the dashboard.
- User navigates to the Sign-In screen, enters their registered email and password, and clicks submit → System verifies the credentials against the Supabase database → System generates a secure session token and redirects the user to the dashboard.
- User clicks the 'Forgot Password' link and submits their email address → System verifies if the email exists and sends a secure, time-limited password reset link → User clicks the link to define and save a new password.
- User attempts to sign up with an email address that is already registered → System rejects the registration and displays an error message prompting the user to log in instead.

### 4.2.3 Functional Requirements

| Req. ID   | Requirement Description                                                                                   | Priority |
|-----------|-----------------------------------------------------------------------------------------------------------|----------|
| REQ-ER-01 | The system SHALL allow users to register a new account using a valid email address and a secure password. | High     |

| Req. ID   | Requirement Description                                                                                                                                                   | Priority |
|-----------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|
| REQ-ER-02 | The system SHALL allow users to log in using their registered email address and password.                                                                                 | High     |
| REQ-ER-03 | The system SHALL securely hash and salt all user passwords using Supabase Auth before storing them in the database; passwords SHALL NEVER be stored in plaintext.         | High     |
| REQ-ER-04 | The system SHALL enforce minimum password complexity rules (e.g., minimum 8 characters) during the registration process..                                                 | Medium   |
| REQ-ER-05 | The system SHALL provide a 'Forgot Password' mechanism that sends a secure password reset link to the user's registered email address.                                    | Medium   |
| REQ-ER-06 | The system SHALL prevent the creation of duplicate accounts using an email address that is already registered in the system.                                              | High     |
| REQ-ER-07 | The system SHALL seamlessly integrate email-authenticated users into the same database schema as OAuth-authenticated users, generating a unique user UUID for both types. | Medium   |

## 4.3 Project Statistics Dashboard

**Priority:** High

**Benefit:** 8 | **Penalty:** 7 | **Cost:** 4 | **Risk:** 2

### 4.3.1 Description

The Project Stats feature provides a visual dashboard of quantitative metrics derived from Git commit history. It gives users a data-backed overview of their project's activity, language composition, and development velocity. Charts are rendered using Recharts.

### 4.3.2 Stimulus/Response Sequences

- User selects a linked repository and navigates to 'Project Stats' → System fetches commit history via PyDriller → Statistics computed → Charts rendered in browser.
- User changes the date-range filter → System re-fetches and recomputes metrics → Charts update dynamically.

## 4.2.3 Functional Requirements

| Req. ID   | Requirement Description                                                                                | Priority |
|-----------|--------------------------------------------------------------------------------------------------------|----------|
| REQ-PS-01 | The system SHALL display a commit frequency chart (bar chart) showing commits per day, week, or month. | High     |

| Req. ID   | Requirement Description                                                                                                                                                     | Priority |
|-----------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|
| REQ-PS-02 | The system SHALL display a language distribution chart (pie/doughnut) showing percentage of code by programming language.                                                   | High     |
| REQ-PS-03 | The system SHALL display a pull request velocity line chart showing PRs opened and merged over time.                                                                        | Medium   |
| REQ-PS-04 | The system SHALL display summary metric cards: total commits, total contributors, most active day, and top language.                                                        | High     |
| REQ-PS-05 | The system SHALL allow users to filter the statistics view by a custom date range.                                                                                          | Medium   |
| REQ-PS-06 | The system SHALL use PyDriller to extract all commit metadata (author, timestamp, lines added/deleted) from the local Git clone.                                            | High     |
| REQ-PS-07 | Statistics SHALL be cached in Supabase after initial computation to avoid redundant re-processing.                                                                          | Medium   |
| REQ-RM-01 | The system SHALL provide a manual "Sync Now" button on the dashboard to fetch the latest commits and update all analysis metrics.                                           | High     |
| REQ-RM-02 | When a user unlinks a repository, the system SHALL permanently delete or archive all cached analysis data, embeddings, and feature results associated with that repository. | Medium   |

## 4.4 Timeline Narration

**Priority:** High

**Benefit:** 9 | **Penalty:** 8 | **Cost:** 7 | **Risk:** 4

### 4.4.1 Description

The Timeline Narration feature uses an LLM (via OpenRouter API) to transform raw, chronological commit data into a readable, story-like narrative. Rather than presenting raw commit messages, the system synthesizes a developer-friendly story such as: "In January, the team laid the foundation by setting up the project scaffold and implementing core authentication. By March, the focus shifted to the payment gateway..."

### 4.4.2 Stimulus/Response Sequences

- User navigates to 'Timeline Narration' → System fetches commit data grouped by time period → Sends structured prompt to OpenRouter LLM API → Streams narrative response to the frontend → User reads the story.
- User clicks a specific story segment → System highlights the corresponding commits in a sidebar.

### 4.4.3 Functional Requirements

| Req. ID   | Requirement Description                                                                                                        | Priority |
|-----------|--------------------------------------------------------------------------------------------------------------------------------|----------|
| REQ-TN-01 | The system SHALL group commits by configurable time periods (weekly or monthly) and generate an AI narrative for each group.   | High     |
| REQ-TN-02 | The system SHALL stream the narrative to the frontend using Server-Sent Events or WebSockets for a progressive display effect. | Medium   |
| REQ-TN-03 | The narration SHALL include references to major features, milestones, and refactoring phases identified from commit messages.  | High     |
| REQ-TN-04 | The system SHALL display the narrative alongside a D3.js-rendered interactive timeline showing date markers.                   | Medium   |
| REQ-TN-05 | The user SHALL be able to regenerate the narration to get a fresh AI response.                                                 | Low      |
| REQ-TN-06 | Generated narratives SHALL be stored in Supabase (NarrationResult table) to avoid regeneration on every view.                  | Medium   |

## 4.5 Collaborator Categorization and Awards

**Priority:** Medium

**Benefit:** 7 | **Penalty:** 5 | **Cost:** 5 | **Risk:** 3

### 4.5.1 Description

This feature analyzes individual contributor behavior patterns from commit data and categorizes each developer into a behavioral role (e.g., 'The Feature Lead', 'The Refactorer', 'The Night Owl', 'The Bug Fixer'). It also computes a rank score for each contributor and awards badges based on preset metrics.

### 4.5.2 Stimulus/Response Sequences

- User navigates to 'Collaborators' → System retrieves contributor data from GitMiningService → Computes role categories and rank scores → Displays contributor cards with badges and rankings.
- User clicks on a specific contributor → System shows individual award details and contribution breakdown.

### 4.5.3 Functional Requirements

| Req. ID   | Requirement Description                                                                                                      | Priority |
|-----------|------------------------------------------------------------------------------------------------------------------------------|----------|
| REQ-CA-01 | The system SHALL extract per-contributor metrics: total commits, lines added, lines deleted, files changed, and active days. | High     |

| Req. ID   | Requirement Description                                                                                                            | Priority |
|-----------|------------------------------------------------------------------------------------------------------------------------------------|----------|
| REQ-CA-02 | The system SHALL classify each contributor into one of at least four behavioral roles using rule-based logic on extracted metrics. | High     |
| REQ-CA-03 | The system SHALL compute a rank score for each contributor using a weighted formula of commit count, lines added, and active days. | Medium   |
| REQ-CA-04 | The system SHALL display contributor cards with: avatar, username, role badge, rank, commit count, and awarded badges.             | Medium   |
| REQ-CA-05 | The system SHALL support viewing individual award details and the criteria for each badge.                                         | Low      |
| REQ-CA-06 | Collaborator data SHALL be stored in the Collaborator entity in Supabase and refreshed on each repository sync.                    | Medium   |

## 4.6 AI Code Review

**Priority:** High

**Benefit:** 9 | **Penalty:** 7 | **Cost:** 7 | **Risk:** 5

### 4.6.1 Description

The AI Code Review feature sends commit diffs to the OpenRouter LLM API for automated analysis. The system produces a health score for the codebase, identifies potential bugs, lists code smell patterns, and provides actionable suggestions. Results are displayed in an organized report with severity levels.

### 4.6.2 Stimulus/Response Sequences

- User navigates to 'Code Review' and clicks 'Run Review' → System fetches recent commit diffs → Sends batched diffs to OpenRouter LLM → LLM returns issues, suggestions, and health score → System renders the report.
- User filters issues by severity (Critical / Warning / Info) → Display updates to show filtered list.

### 4.6.3 Functional Requirements

| Req. ID   | Requirement Description                                                                                            | Priority |
|-----------|--------------------------------------------------------------------------------------------------------------------|----------|
| REQ-CR-01 | The system SHALL send commit diffs (up to the N most recent commits) to the LLM API for code review.               | High     |
| REQ-CR-02 | The system SHALL compute and display a numerical codebase health score (0–100) based on LLM analysis results.      | High     |
| REQ-CR-03 | The system SHALL list identified issues with a severity level (Critical, Warning, Info) and a file/line reference. | High     |
| REQ-CR-04 | The system SHALL display actionable suggestions for each identified issue.                                         | High     |
| REQ-CR-05 | The user SHALL be able to filter the issue list by severity level.                                                 | Medium   |
| REQ-CR-06 | The user SHALL be able to view the number of reviewed commits in the current analysis.                             | Low      |
| REQ-CR-07 | Code review results SHALL be stored in the CodeReviewResult entity in Supabase.                                    | Medium   |
| REQ-CR-08 | The system SHALL check for ownership (REQ-AU-06) before allowing a code review to be run.                          | High     |

## 4.7 RAG-Based Repository Querying

**Priority:** High

**Benefit:** 9 | **Penalty:** 8 | **Cost:** 8 | **Risk:** 6

### 4.7.1 Description

The RAG (Retrieval-Augmented Generation) feature provides a natural language chat interface that allows users to ask questions about their codebase. The system embeds the repository's source files into a vector database (Pinecone/ChromaDB) and at query time retrieves the most semantically relevant code chunks to provide context-aware answers via the LLM.

### 4.7.2 Stimulus/Response Sequences

- Repository is linked → System triggers embedding: code files are split into chunks, embedded using LLM embeddings, and stored in Pinecone/ChromaDB.
- User types a question in the chat box (e.g., 'How does the login flow work?') → System embeds the query → Retrieves top-K relevant code chunks from vector DB → Sends chunks + query to LLM → Streams answer back to the chat interface.
- User asks a follow-up question → Previous conversation history is included in the prompt for context continuity.

### 4.7.3 Functional Requirements

| Req. ID   | Requirement Description                                                                                                   | Priority |
|-----------|---------------------------------------------------------------------------------------------------------------------------|----------|
| REQ-RG-01 | The system SHALL embed the full codebase into a vector database upon repository linking or sync.                          | High     |
| REQ-RG-02 | The system SHALL accept free-text questions from the user in a chat interface.                                            | High     |
| REQ-RG-03 | The system SHALL retrieve the top-K (configurable, default K=5) most semantically relevant code chunks for each query.    | High     |
| REQ-RG-04 | The retrieved context and the user query SHALL be combined into a structured prompt and sent to the LLM API.              | High     |
| REQ-RG-05 | The system SHALL display the LLM response in a conversational message bubble format.                                      | High     |
| REQ-RG-06 | The system SHALL maintain a RAGSession record including the session ID, message history, vector index ID, and start time. | Medium   |
| REQ-RG-07 | The RAG session SHALL support multi-turn conversations by including prior message history in each prompt.                 | Medium   |
| REQ-RG-08 | The system SHALL provide a 'Clear Session' button to start a new conversation.                                            | Low      |
| REQ-RG-09 | The system SHALL allow users to view a history of their past RAG chat sessions.                                           | Low      |
| REQ-RG-10 | The system SHALL allow users to delete individual RAG chat sessions from their history.                                   | Low      |

## 4.8 Auto-Documentation

**Priority:** Medium

**Benefit:** 8 | **Penalty:** 5 | **Cost:** 6 | **Risk:** 4

### 4.8.1 Description

Auto-Documentation parses the repository's code structure and uses the LLM to generate and maintain documentation automatically. This includes a project-level README, module summaries, and individual function-level documentation. The output is presented as Markdown that can be copied or exported.

### 4.8.2 Stimulus/Response Sequences

- User navigates to 'Documentation' → System parses code AST/structure → Sends structured prompt per module to LLM → Aggregates results → Displays formatted documentation.
- User exports the generated README → System renders it as downloadable Markdown.

### 4.8.3 Functional Requirements

| Req. ID   | Requirement Description                                                                                          | Priority |
|-----------|------------------------------------------------------------------------------------------------------------------|----------|
| REQ-AD-01 | The system SHALL parse the repository structure to identify modules, classes, and functions.                     | High     |
| REQ-AD-02 | The system SHALL generate a README.md containing: project overview, setup instructions, and module descriptions. | High     |
| REQ-AD-03 | The system SHALL generate per-function summaries describing input parameters, return values, and behavior.       | Medium   |
| REQ-AD-04 | The system SHALL generate a module map (JSON) showing the hierarchical structure of the codebase.                | Medium   |
| REQ-AD-05 | Generated documentation SHALL be stored in the DocumentationResult entity in Supabase.                           | Medium   |
| REQ-AD-06 | The user SHALL be able to export the generated README as a downloadable .md file.                                | Medium   |
| REQ-AD-07 | The system SHALL regenerate documentation when the repository is re-synced.                                      | Low      |

## 4.9 File Hotzones (Heatmap)

**Priority:** Medium

**Benefit:** 7 | **Penalty:** 5 | **Cost:** 5 | **Risk:** 3

### 4.9.1 Description

The File Hotzones feature uses PyDriller to compute a churn score for each file in the repository (number of times modified) and visualizes it as a D3.js Treemap. Files are color-coded from green (stable) to red (high churn/instability), helping developers instantly identify risky areas of the codebase.

### 4.9.2 Stimulus/Response Sequences

- User navigates to 'Hotzones' → System computes churn score per file using PyDriller → D3.js Treemap rendered with color gradient → User can hover over a file to see churn details.
- User clicks a file in the Treemap → System shows commit history for that file and its hotzone level.

### 4.9.3 Functional Requirements



| Req. ID   | Requirement Description                                                                                                        | Priority |
|-----------|--------------------------------------------------------------------------------------------------------------------------------|----------|
| REQ-HZ-01 | The system SHALL compute a churn score for each file as the total number of commits modifying that file.                       | High     |
| REQ-HZ-02 | The system SHALL render a D3.js Treemap where each rectangle represents a file, sized proportionally to churn score.           | High     |
| REQ-HZ-03 | Files SHALL be color-coded on a green-yellow-red gradient: green = low churn, red = high churn.                                | High     |
| REQ-HZ-04 | Hovering over a file rectangle SHALL display a tooltip showing: file path, churn score, total commits, and last modified date. | Medium   |
| REQ-HZ-05 | Clicking a file SHALL display its commit history in a side panel.                                                              | Low      |
| REQ-HZ-06 | File hotzone data SHALL be stored in the FileRecord entity in Supabase.                                                        | Medium   |
| REQ-HZ-07 | The hotzone view SHALL support filtering by file language.                                                                     | Low      |

## 5. Other Non-Functional Requirements

### 5.1 Performance Requirements

| Metric                      | Target                 | Condition                                  |
|-----------------------------|------------------------|--------------------------------------------|
| Page load time              | < 2 seconds            | For dashboard with cached data             |
| Initial repository analysis | < 5 minutes            | For repos with up to 1,000 commits         |
| RAG query response start    | < 5 seconds            | First token from LLM must appear within 3s |
| API response time (non-AI)  | < 2 seconds            | For project stats and collaborator data    |
| Concurrent users            | 10+ simultaneous users | Without performance degradation            |

### 5.2 Safety Requirements

GitStory is an analysis and visualization tool and does not directly control any physical processes. However, the following safety-related concerns apply:

- The system SHALL NOT modify or write back to any user's GitHub repository. All operations are read-only on the Git data.
- AI-generated outputs (code review suggestions, narratives, documentation) are advisory only and SHALL be clearly labeled as AI-generated content.

- The system SHALL display a disclaimer that AI Code Review results may contain inaccuracies and should not be used as the sole basis for security or production decisions.

## 5.3 Security Requirements

- REQ-SEC-01: All data in transit SHALL be encrypted using TLS 1.2 or higher.
- REQ-SEC-02: GitHub OAuth access tokens SHALL be stored server-side in Supabase and NEVER exposed to the client-side JavaScript.
- REQ-SEC-03: All API endpoints SHALL require a valid, non-expired session token (Bearer token) from Supabase Auth.
- REQ-SEC-04: The system SHALL prevent unauthorized users from accessing analysis results of repositories they do not own.
- REQ-SEC-05: API keys for OpenRouter and Pinecone SHALL be stored as environment variables and never committed to source control.
- REQ-SEC-06: The system SHALL implement input validation and sanitization on all user-supplied text (e.g., repository URLs, RAG queries) to prevent injection attacks.

## 5.4 Software Quality Attributes

| Quality Attribute      | Description and Target                                                                                                                                                                          |
|------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Usability</b>       | Non-technical users (instructors) should be able to understand the dashboard within 5 minutes with no training. A usability rating of 4/5 is the target.                                        |
| <b>Reliability</b>     | The system should achieve 99% uptime for core features. AI features may have degraded availability when the external LLM API is down.                                                           |
| <b>Maintainability</b> | The codebase should follow modular design with clear separation between analysis engine, API layer, and frontend. New analysis features should be addable without redesigning existing modules. |
| <b>Scalability</b>     | The backend should be horizontally scalable. Repository analysis jobs should be queueable to handle burst loads.                                                                                |
| <b>Portability</b>     | The system should be deployable on any Linux-based cloud platform (AWS, GCP, Azure, Railway) with Docker support.                                                                               |
| <b>Testability</b>     | All API endpoints shall have unit and integration tests achieving at least 70% code coverage.                                                                                                   |

## 5.5 Business Rules

- A user must be logged in via GitHub OAuth to access any feature beyond the landing page.
- A user can only analyze repositories that they own or have been granted explicit access to.

- AI features (narration, code review, documentation, RAG) require a repository to have been fully indexed first.
- Collaborator awards are only computed for repositories with at least 2 contributors.
- RAG sessions are user-specific and private; one user cannot view another user's chat.

## 6. Other Requirements

---

### 6.1 Database Requirements

The system uses PostgreSQL (via Supabase) as its primary relational database. The core entities and their relationships are defined in the Design Class Diagram in Appendix B. Key tables include: User, Repository, Project, CommitRecord, FileRecord, Collaborator, NarrationResult, CodeReviewResult, DocumentationResult, FeatureResult, and RAGSession.

- All primary keys SHALL use UUIDs (UUID v4).
- Foreign key constraints SHALL be enforced at the database level.
- Sensitive fields (OAuth tokens) SHALL NOT be stored in plaintext.

### 6.2 Internationalization

Version 1.0 will support English only. Future versions may support additional languages for the UI. AI-generated content (narration, code review) will be produced in the language of the prompt, defaulting to English.

### 6.3 Legal and Compliance

- The system must comply with GitHub's Terms of Service for API usage.
- User data collection must comply with relevant privacy regulations (GDPR for EU users). A privacy policy must be published before public release.
- OpenRouter API usage must comply with the respective LLM providers' usage policies.

## Appendix A: Glossary

| Term / Acronym      | Definition                                                                                                                                       |
|---------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Git</b>          | A distributed version control system used for tracking changes in source code.                                                                   |
| <b>GitStory</b>     | The system described in this SRS — an AI-powered Git repository analysis and narration platform.                                                 |
| <b>RAG</b>          | Retrieval-Augmented Generation — a technique where an LLM is given retrieved context documents to produce more accurate, grounded answers.       |
| <b>LLM</b>          | Large Language Model — an AI model trained on large amounts of text data, capable of generating natural language and code (e.g., GPT-4, Claude). |
| <b>OAuth 2.0</b>    | An authorization framework that allows third-party applications (GitStory) to obtain limited access to a user's account on GitHub.               |
| <b>PyDriller</b>    | An open-source Python library for mining Git repositories, providing commit history, file changes, and author metrics.                           |
| <b>Churn Score</b>  | A metric representing how frequently a file has been modified across all commits. High churn indicates instability.                              |
| <b>Hotzone</b>      | A file or module identified as high-risk due to excessive modification frequency (high churn score).                                             |
| <b>Supabase</b>     | An open-source Firebase alternative providing a hosted PostgreSQL database, authentication, and real-time subscriptions.                         |
| <b>Pinecone</b>     | A cloud-hosted vector database used for storing and querying code embeddings in the RAG pipeline.                                                |
| <b>DFD</b>          | Data Flow Diagram — a diagrammatic representation of data flow through a system or process.                                                      |
| <b>SRS</b>          | Software Requirements Specification — this document.                                                                                             |
| <b>UUID</b>         | Universally Unique Identifier — a 128-bit label used to uniquely identify records in the database.                                               |
| <b>FastAPI</b>      | A modern, high-performance Python web framework for building REST APIs.                                                                          |
| <b>Treemap</b>      | A type of data visualization that displays hierarchical data as nested rectangles, sized by a metric (here: churn score).                        |
| <b>Embedding</b>    | A numerical vector representation of text or code used for semantic similarity search in vector databases.                                       |
| <b>Health Score</b> | A numerical score (0–100) representing the overall code quality of a repository, computed from AI Code Review results.                           |

## Appendix B: Analysis Models

### B.1 Design Class Diagram

The Design Class Diagram below shows the complete object model for the GitStory system including all entities, their attributes, methods, and relationships. The diagram is organized by domain layer: User/Auth (blue), Repository (orange), Project/Analysis (yellow-green), Git Data (purple), Feature Results (teal), Service Layer (dark green), and AI/External (red).



Figure B.1: GitStory Design Class Diagram — Full UML class diagram showing entities (User, Repository, Project, CommitRecord, FileRecord, Collaborator, FeatureResult, NarrationResult, CodeReviewResult, DocumentationResult, RAGSession), service classes (GitMiningService, AnalysisOrchestrator), and their relationships.

Key relationships shown in the Design Class Diagram:

- User (1) → owns → (\*) Repository: A user may link multiple GitHub repositories.
- Repository (1) → has → (1) Project: Each linked repository has one analysis project.
- Project (1) → contains → (\*) CommitRecord: A project aggregates all commit records from Git history.
- Project (1) → contains → (\*) FileRecord: File-level churn data belongs to a project.
- Project (1) → has → (\*) Collaborator: Multiple developers contribute to each project.
- Project (1) → stores → (\*) FeatureResult: Analysis outputs (narration, code review, docs) are stored as feature results.
- FeatureResult → specializes into → NarrationResult, CodeReviewResult, DocumentationResult.
- RAGSession → belongs to → User: Chat sessions are user-specific.
- AnalysisOrchestrator (service) → orchestrates → NarrationResult, CodeReviewResult, DocumentationResult, FeatureResult.

- GitMiningService (service) → mines → Repository: Uses PyDriller to extract raw Git data.

## B.2 DFD Level 1 — Detailed Data Flow Diagram

The DFD Level 1 diagram expands the Context Diagram by decomposing the GitStory system into its three major processing subsystems: Authentication & Profile Management, Analysis Engine, and Intelligent Queries. It shows the data flows between these processes, external entities (User, GitHub, LLM API, Vector Database), and the data stores (User & Auth Database, Repo Data Source).

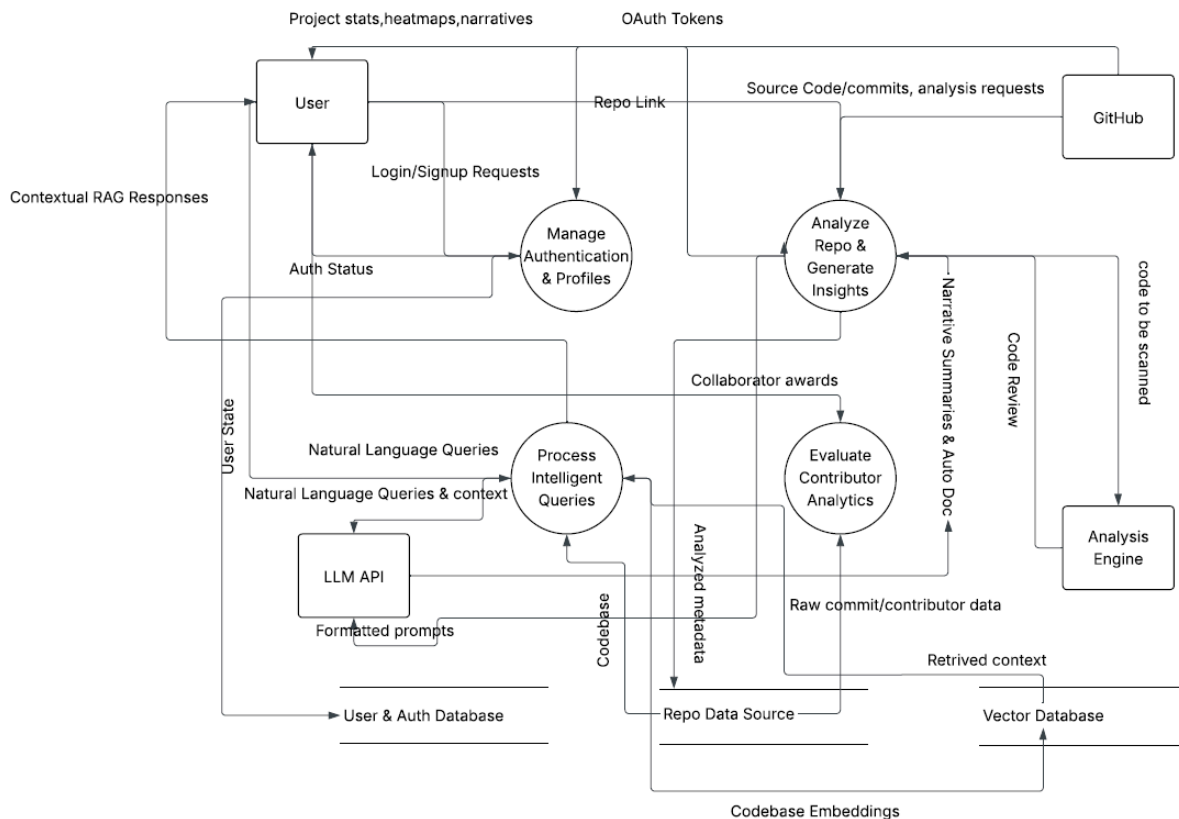


Figure B.2: GitStory DFD Level 1 — Decomposition of the system into three major processes: (1) Manage Authentication & Profiles, (2) Analyze Repo & Generate Insights, and (3) Intelligent Queries. Shows all inter-process data flows, data stores, and external entity connections.

The three Level-1 processes and their key data flows are:

- **Process 1 — Manage Authentication & Profiles:** Receives login/signup requests from the User; exchanges OAuth tokens with GitHub; reads/writes the User & Auth Database; sends auth status back to the User.
- **Process 2 — Analyze Repo & Generate Insights:** Receives the repository link from the authenticated User; fetches source code and commits from GitHub; sends analysis requests and receives narrative summaries, auto-documentation, and code review results from the LLM API; stores and retrieves analyzed metadata from the Repo Data Source; sends project stats, heatmaps, and narratives back to the User.

- Process 3 — Intelligent Queries (RAG): Receives natural language queries from the User; sends codebase embeddings to the Vector Database and retrieves context; sends formatted prompts to the LLM API and receives contextual RAG responses; delivers answers and collaborator awards back to the User.

## Appendix C: To Be Determined (TBD) List

---

| # | TBD Item                                                                                            | Associated Section | Status |
|---|-----------------------------------------------------------------------------------------------------|--------------------|--------|
| 1 | Exact LLM model selection on OpenRouter for production (cost vs quality trade-off).                 | 3.3                | Open   |
| 2 | Vector database final choice between Pinecone and ChromaDB — cost and latency benchmarking pending. | 3.3                | Open   |
| 3 | Final weighting formula for Collaborator rank score computation.                                    | 4.4                | Open   |
| 4 | Maximum number of commits to include in a single Code Review batch (LLM token limit constraint).    | 4.5                | Open   |
| 5 | User manual PDF format and delivery mechanism.                                                      | 2.6                | Open   |
| 6 | GDPR privacy policy text and data retention policy.                                                 | 6.3                | Open   |
| 7 | Deployment platform (Railway, Vercel, AWS EC2) for production v1.0.                                 | 2.4                | Open   |